

Estructura del proyecto FastKote

Proyecto: FastKote - Sistema de generación de cotizaciones

Cuaderno: Construcción e implementación

Versión analizada: fastkote-fullstack-layered-v2 (2).zip

1. Propósito del documento

Este documento describe la estructura real del proyecto FastKote en código. Sirve como fuente para el Cuaderno 3 de NotebookLM porque permite explicar cómo está organizado el frontend, el backend, la base de datos y la documentación técnica.

La estructura actual evidencia una separación por responsabilidades, evitando una implementación monolítica en un solo archivo. Esta organización apoya la mantenibilidad solicitada por el SRS y permite relacionar módulos construidos con RF/CU concretos.

2. Estructura raíz del proyecto

```
fastkote-fullstack-layered/  
├─ backend/  
├─ frontend/  
├─ database/  
├─ docs/  
├─ docker-compose.yml  
└─ README.md
```

Carpeta / archivo	Responsabilidad
backend/	API REST, lógica de negocio, patrones, seguridad, persistencia y servicios externos.

Carpeta / archivo	Responsabilidad
frontend/	Interfaz de usuario, páginas, componentes y consumo de API.
database/	Scripts SQL para crear y poblar PostgreSQL.
docs/	Documentación de arquitectura, patrones y casos de uso implementados.
docker- compose.yml	Contenedor PostgreSQL para desarrollo local.
README.md	Instrucciones de instalación, ejecución y alcance implementado.

3. Estructura del backend

```
backend/  
├─ prisma/  
│   └─ schema.prisma  
├─ src/  
│   ├── application/  
│   ├── domain/  
│   ├── infrastructure/  
│   ├── interfaces/  
│   └─ shared/  
├─ package.json  
└─ tsconfig.json
```

El backend implementa una arquitectura por capas. Cada capa tiene una responsabilidad definida.

4. Capa interfaces/http

```
backend/src/interfaces/http/  
├─ controllers/  
│   ├─ auth.controller.ts  
│   ├─ calendar.controller.ts  
│   ├─ catalog.controller.ts  
│   ├─ clients.controller.ts  
│   ├─ employees.controller.ts  
│   ├─ quotes.controller.ts  
│   └─ roles.controller.ts  
├─ middlewares/  
│   ├─ auth.middleware.ts  
│   └─ error.middleware.ts  
├─ createMediator.ts  
└─ server.ts
```

Responsabilidad

Esta capa recibe solicitudes HTTP desde el frontend, aplica autenticación/autorización, llama al Mediator y devuelve respuestas JSON o archivos PDF.

Controladores implementados

Controlador	Módulo	RF/CU relacionado
auth.controller.ts	Login	RF-01 / CU-SGC-Lo-01.
clients.controller.ts	Clientes	RF-03 / CU-SGC-Lo-04.
employees.controller.ts	Empleados	RF-06 / CU-SGC-Lo-05.
roles.controller.ts	Roles	RF-06 / CU-SGC-L1-05 / CU-SGC-L1-06.
quotes.controller.ts	Cotizaciones, PDF, WhatsApp	RF-02 / CU-SGC-Lo-03 / CU-SGC-L1-18 / CU-SGC-L1-19.
calendar.controller.ts	Calendario	CU-SGC-L1-17.

Controlador	Módulo	RF/CU relacionado
catalog.controller.ts	Paquetes activos	RF-04 / CU-SGC-Lo-07.

5. Capa application

```
backend/src/application/  
├─ auth/  
├─ clients/  
├─ employees/  
├─ mediator/  
└─ quotes/
```

Responsabilidad

Esta capa contiene los casos de uso implementados como handlers. Aquí se aplica la lógica de negocio del sistema y los patrones Mediator y Strategy.

6. Casos de uso de autenticación

```
backend/src/application/auth/  
└─ LoginUser.handler.ts
```

Handler	Responsabilidad	RF/CU
LoginUserHandler	Validar credenciales, intentos fallidos, empleado activo, roles y generar JWT.	RF-01 / CU-SGC-Lo-01.

7. Casos de uso de clientes

```
backend/src/application/clients/  
├─ CreateClient.handler.ts  
├─ GetClient.handler.ts  
├─ ListClients.handler.ts  
└─ UpdateClient.handler.ts
```

Handler	Responsabilidad	RF/CU
CreateClientHandler	Registrar nuevo cliente.	RF-03 / CU-SGC-L1-07.
GetClientHandler	Consultar detalle de cliente.	RF-03 / CU-SGC-L1-09.
ListClientsHandler	Listar y filtrar clientes.	RF-03 / CU-SGC-L1-09.
UpdateClientHandler	Actualizar información de cliente.	RF-03 / CU-SGC-L1-08.

8. Casos de uso de empleados y roles

```
backend/src/application/employees/  
├─ AssignRoles.handler.ts  
├─ CreateEmployee.handler.ts  
├─ DeactivateEmployee.handler.ts  
├─ ListEmployees.handler.ts  
├─ ListRoles.handler.ts  
└─ UpdateEmployee.handler.ts
```

Handler	Responsabilidad	RF/CU
CreateEmployeeHandler	Registrar empleado.	RF-06 / CU-SGC-L1-01.
UpdateEmployeeHandler	Editar empleado.	RF-06 / CU-SGC-L1-02.
ListEmployeesHandler	Consultar empleados.	RF-06 / CU-SGC-L1-03.

Handler	Responsabilidad	RF/CU
DeactivateEmployeeHandler	Desactivar empleado.	RF-06 / CU-SGC-L1-04.
AssignRolesHandler	Asignar roles de sistema.	RF-06 / CU-SGC-L1-05.
ListRolesHandler	Consultar roles.	RF-06 / CU-SGC-L1-06.

9. Casos de uso de cotizaciones

```
backend/src/application/quotes/  
├─ CreateQuote.handler.ts  
├─ GenerateQuotePdf.handler.ts  
├─ ListCalendar.handler.ts  
├─ ListPackages.handler.ts  
├─ ListQuotes.handler.ts  
├─ SendQuoteWhatsApp.handler.ts  
├─ UpdateQuote.handler.ts  
├─ UpdateQuoteStatus.handler.ts  
└─ strategies/
```

Handler	Responsabilidad	RF/CU
ListQuotesHandler	Consultar cotizaciones con filtros.	RF-02 / CU-SGC-L1-14.
CreateQuoteHandler	Crear cotización y calcular valores.	RF-02 / CU-SGC-L1-11.
UpdateQuoteHandler	Editar cotización en borrador e incrementar versión.	RF-02 / CU-SGC-L1-13.
UpdateQuoteStatusHandler	Cambiar estado y aplicar estrategia de calendario.	RF-02 / CU-SGC-L1-15.

Handler	Responsabilidad	RF/CU
GenerateQuotePdfHandler	Generar PDF de cotización.	CU-SGC-L1-18.
SendQuoteWhatsAppHandler	Enviar cotización por WhatsApp.	CU-SGC-L1-19.
ListCalendarHandler	Consultar eventos bloqueados.	CU-SGC-L1-17.
ListPackagesHandler	Consultar paquetes activos.	RF-04 / CU-SGC-Lo-07.

10. Patrón Mediator en la estructura

```
backend/src/application/mediator/
```

```
└─ Mediator.ts
```

El archivo `Mediator.ts` define una clase que registra handlers mediante claves y los ejecuta con `send(...)`. El archivo `createMediator.ts` instancia repositorios, servicios, contextos de Strategy y registra todos los handlers.

Flujo:

```
Controller → Mediator → Handler → Repository / Service
```

Este patrón se aplica en RF-01, RF-02, RF-03 y RF-06.

11. Patrón Strategy en la estructura

```
backend/src/application/quotes/strategies/
```

```
├─ pricing/
```

```
└─ status/
```

Strategy de precios

```
pricing/  
├─ CustomPricingStrategy.ts  
├─ FixedPackagePricingStrategy.ts  
├─ PerChildPricingStrategy.ts  
├─ PricingContext.ts  
└─ PricingStrategy.ts
```

Se relaciona con **RF-02, CU-SGC-L1-11 Generar Cotización** y **CU-SGC-L1-12 Generar Cotización Personalizada**.

Strategy de estado

```
status/  
├─ AcceptQuoteStatusStrategy.ts  
├─ DraftOrSentStatusStrategy.ts  
├─ QuoteStatusContext.ts  
├─ QuoteStatusStrategy.ts  
└─ ReleaseQuoteStatusStrategy.ts
```

Se relaciona con **CU-SGC-L1-15 Actualizar Estado Cotización** y **CU-SGC-L1-17 Consultar Calendario de Eventos**.

12. Capa domain

```
backend/src/domain/repositories/  
├─ AuthRepository.ts  
├─ ClientRepository.ts  
├─ EmployeeRepository.ts  
└─ QuoteRepository.ts
```

Responsabilidad

La capa de dominio define contratos de repositorios. Estos contratos indican qué operaciones necesita la aplicación sin depender directamente de Prisma, Express o PostgreSQL.

Contrato	Módulo
AuthRepository	Autenticación.
ClientRepository	Clientes.
EmployeeRepository	Empleados y roles.
QuoteRepository	Cotizaciones, paquetes y calendario.

13. Capa infrastructure

```
backend/src/infrastructure/  
├─ repositories/  
└─ services/
```

Repositorios Prisma

```
repositories/  
├─ PrismaAuthRepository.ts  
├─ PrismaClientRepository.ts  
├─ PrismaEmployeeRepository.ts  
└─ PrismaQuoteRepository.ts
```

Estos archivos implementan los contratos de dominio usando Prisma ORM y PostgreSQL.

Servicios de infraestructura

```
services/  
├─ JwtService.ts  
└─ PasswordHasher.ts
```

└─ PdfQuoteService.ts

└─ WhatsAppGateway.ts

Servicio	Responsabilidad	RF/CU
JwtService	Firmar y verificar tokens.	RF-01.
PasswordHasher	Hash y comparación de contraseñas.	RF-01.
PdfQuoteService	Generar PDF de cotización.	CU-SGC-L1-18.
WhatsAppGateway	Enviar o simular envío por WhatsApp.	CU-SGC-L1-19.

14. Capa shared

backend/src/shared/

└─ config/

└─ errors/

└─ prisma/

Carpeta	Responsabilidad
config/	Variables de entorno del sistema.
errors/	Error HTTP personalizado.
prisma/	Instancia compartida de Prisma Client.

15. Estructura del frontend

frontend/src/

└─ api/

└─ auth/

```
├─ components/  
├─ pages/  
├─ styles/  
├─ App.tsx  
└─ main.tsx
```

Carpeta api

```
frontend/src/api/  
├─ auth.ts  
├─ clients.ts  
├─ employees.ts  
├─ http.ts  
└─ quotes.ts
```

Responsabilidad: consumir la API REST del backend.

Carpeta auth

```
frontend/src/auth/  
└─ AuthContext.tsx
```

Responsabilidad: mantener sesión, token, usuario autenticado y roles.

Carpeta components

```
frontend/src/components/  
├─ layout/  
├─ public/  
└─ ui/
```

Subcarpeta	Uso
layout/	Layout general y pantalla de login.

Subcarpeta	Uso
public/	Landing pública y página en construcción.
ui/	Componentes reutilizables como badge y modal.

Carpeta pages

```
frontend/src/pages/  
├─ CalendarPage.tsx  
├─ ClientsPage.tsx  
├─ DashboardPage.tsx  
├─ EmployeesPage.tsx  
└─ QuotesPage.tsx
```

Página	RF/CU relacionado
CalendarPage.tsx	CU-SGC-L1-17.
ClientsPage.tsx	RF-03 / CU-SGC-Lo-04.
DashboardPage.tsx	Acceso principal por rol.
EmployeesPage.tsx	RF-06 / CU-SGC-Lo-05.
QuotesPage.tsx	RF-02 / CU-SGC-Lo-03.

16. Estructura de base de datos

```
database/  
├─ 01_schema.sql  
└─ 02_seed.sql
```

Archivo	Uso
01_schema.sql	Crea tipos ENUM, tablas, claves foráneas e índices.
02_seed.sql	Inserta datos base como roles, usuarios, empleados y paquetes.

También existe el modelo Prisma:

```
backend/prisma/schema.prisma
```

Este archivo define el modelo ORM usado por los repositorios.

17. Conclusión

La estructura del proyecto FastKote evidencia una construcción por capas, con separación entre frontend, controladores HTTP, aplicación, dominio, infraestructura y base de datos. Esta organización permite defender técnicamente la implementación porque cada módulo se puede relacionar con un RF/CU del SRS. Además, la presencia de Mediator y Strategy demuestra que se aplicaron patrones de diseño para reducir acoplamiento y mejorar la mantenibilidad del sistema.